Frontend/Backend Assignment - Document Intelligence Platform

To grab the internship, you are required to complete a small assignment. Early submissions are given preference.

In this task, you are required to create a full-stack web application with AI/RAG integration.

Objective:

Build a **full-stack web application with AI integration** that processes book data and enables intelligent querying.

The system should:

- Collect book data using automation
- Store and manage it in a backend
- Generate AI-based insights
- Support question-answering over books
- Expose functionality via REST APIs
- Provide a simple frontend interface

Backend (Django REST Framework)

GET APIs:

- Lists all uploaded books from the database

- Retrieves all detail about each book

- Recommends related books

POST APIs:

- Uploading and processing books and details

- Asking questions about books (RAG query endpoint)

Document Processing Engine

Create a python application that scrapes books and provides AI insights.

Extra marks for handling caching, smart chunking strategies, and optimized embedding generation.

1. Uses automation to **collect book data from the web (any site)**
2. Stores it in a backend
3. Uses AI to **generate insights about books**
4. Exposes everything via APIs
5. Displays results on a simple frontend

AI Insight Generation

You must generate **at least 2 of the following**:

- **Summary:** Generate a short summary of the book
- **Genre Classification:** Predict genre based on description
- **Recommendation Logic:** "If you like X, you'll like Y"
- **Sentiment Analysis:** Analyze tone of reviews/description

RAG Pipeline Implementation:

The system should implement a complete RAG pipeline that:

- Generates embeddings for user questions

- Performs similarity search across book chunks

- Constructs relevant context from retrieved chunks

- Generates contextual answers using LLM with source citations

Frontend (ReactJS / NextJS with Tailwind CSS)

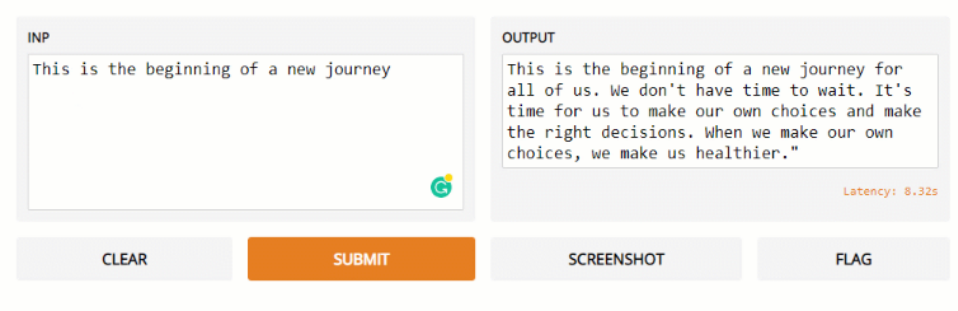
User Interface:

The frontend should be developed using ReactJS or NextJS, styled with Tailwind CSS.

Required Pages:

1. **Dashboard/Book Listing Page**:
    - List all uploaded books with title, author, rating / reviews, description (if available), book URL

2. **Q&A Interface**:
    - Question input
    - Answer display (Display summary / genre / recommendations)



3. **Book detail page**
    - Displays details about the book selected by the user

Database Schema Hints:

Design your database with the following table structure (there can be more tables):

Books table: Store document metadata including title, author, rating / reviews, description (if available), book URL

Tech Stack Requirements:

- **Backend**: Django REST Framework, Python
- **Database**: MySQL for metadata, ChromaDB/FAISS for vectors
- **Frontend**: ReactJS/NextJS with Tailwind CSS
- **AI Integration**: OpenAI API, Claude API, or **LM Studio** (recommended if
- **Automation**: Selenium

AI Integration Options:

Option 1: Use external APIs (OpenAI, Anthropic Claude)

Option 2: Use LM Studio for local LLM hosting (recommended alternative)

LM Studio allows you to run language models locally without requiring external API keys. Download and set up LM Studio with models like Llama, Mistral, or Code Llama for document question-answering tasks. This provides a cost-effective solution and ensures data privacy.

Deadline

18 April, 2026, Saturday, 11:55 PM

Submission Format

- Create a GitHub repository and add your code to it.

- In the README, add:
  - Screenshots of the UI you have created
  - Setup instructions for running the application
  - API documentation
  - Sample questions and answers from your system
- Include a requirements.txt file with all dependencies
- Add samples for testing
- Fill your repo link in this form: https://forms.gle/Fby8pMSmBJqjuVf56.
- Make sure the code is neat and readable with proper comments.
- Make sure the README.md has 3-4 screenshots of the UI

Evaluation Criteria:

- **Functionality** (40%): Working RAG pipeline, accurate answers, proper citations
- **Code Quality** (25%): Clean, readable, well-structured code
- **UI/UX** (20%): User-friendly interface, responsive design
- **Innovation** (15%): Creative features, optimization techniques, error handling

Bonus Points (Optional):

- Caching AI responses (avoid repeated calls)
- Embedding-based similarity (vector search)
- Async processing (Celery / background jobs)
- Bulk scraping pipeline
- Multi-page scraping
- Loading states + UX polish
- Advanced chunking strategies (semantic chunking, overlapping windows)
- Saving chat history

For any doubts, feel free to drop a mail at devgods99@gmail.com We'll try to respond as soon as possible!

In case you're not able to complete it within the deadline, do submit the code even if a part of it doesn't work. The effort and skills you demonstrate through your code matter.

Note: An early submission will give you an advantage over others.

Helpful Resources:

- LM Studio: https://lmstudio.ai/ (for local LLM hosting)
- ChromaDB Documentation: https://docs.trychroma.com/
- Sentence Transformers: https://www.sbert.net/
- Django REST Framework: https://www.django-rest-framework.org/
- React File Upload: https://react-dropzone.js.org/